

Consolidated Peer Evaluation Form

Class Code: 9486

Group #: (RUST)

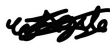
Schedule (Time and Days): WS 8:30 - 9:30 AM and 4:00 - 5:30

Activity: Prelim Project

#	Names of Group Members (family name first) (arranged alphabetically)	Criteria					T	Remarks
		C1	C2	C3	C4	C5		
1	Aguilan, Jecquar Ravent C.	5	5	5	5	5	25	- Group Leader and Project initiator
2	Aoay, Michael Vincent V.	5	5	5	5	5	25	- Helper of the leader
3	Argao, Jeiloyd S.	5	5	5	5	5	25	- Enthusiastic member who gets excited presenting his idea
4	Facun Jared Eizen B.	5	5	5	5	5	25	- A member who loves taking on a challenge
5	Piamonte, Malech Dangyan L.	5	5	5	5	5	25	- Does his work efficiently and on time
6	Talavera, Adrian F.	5	5	5	5	5	25	- Ensures quality output for the project

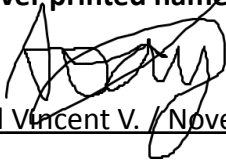
We, the undersigned, attest to the objectivity of the foregoing evaluations.

Evaluators:



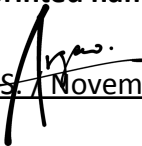
Aguilan, Jecquar Ravent C. / November 11, 2025

[signature(s) over printed name(s)/date signed]



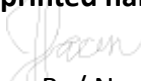
Aoay, Michael Vincent V. / November 27, 2025

[signature(s) over printed name(s)/date signed]



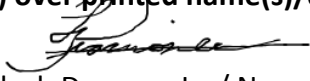
Argao, Jeiloyd S. / November 27, 2025

[signature(s) over printed name(s)/date signed]



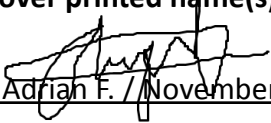
Facun, Jared Eizen B. / November 27, 2025

[signature(s) over printed name(s)/date signed]



Piamonte, Malech Dangyan L. / November 27, 2025

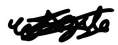
[signature(s) over printed name(s)/date signed]

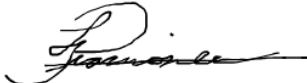
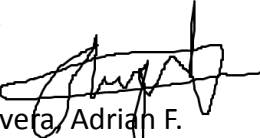


Talavera, Adrian F. / November 27, 2025

[signature(s) over printed name(s)/date signed]

Note: Same % contribution among members is limited to 2 members only. Difference from one % contribution to another must be at least 5.

Member (Signature over Printed Name)	Detailed description of the member’s SIGNIFICANT CONTRIBUTION/S (This should substantiate the % contribution)	% Contribution	Remark
 Aguilan, Jecquar Ravent C.	- This member designed and implemented the Dockerization strategy for the Node.js backend application. They began by authoring the Dockerfile for the backend, carefully selecting the appropriate official Node.js base image to ensure compatibility and optimal performance. They structured the Dockerfile to efficiently copy the application code, run npm install for dependencies, and define the correct command to start the server. Furthermore, they configured the container to expose the correct internal port on which the backend application listens for requests. They rigorously tested the backend container in isolation to verify that it built without errors and ran as expected before integration. This foundational work ensured the backend service was a self-contained, portable unit ready for multi-container orchestration.	14%	VS
 Aoay, Michael Vincent V.	- This member architected the web-serving layer by creating the docker.nginx configuration to act as a reverse proxy. They designed and wrote the specific Nginx configuration file, defining an upstream directive for the backend service and a server block to correctly route API requests to it. They also configured the server block to serve the static frontend files from the designated volume, ensuring efficient delivery of the client-side application. This design effectively unified the frontend and backend, presenting a single entry point to the user's browser and resolving potential CORS issues. They selected the official Nginx Alpine image for its small footprint and security, tailoring it with their custom configuration. This reverse proxy setup was critical for enabling the frontend and backend to communicate seamlessly as if they were a single application.	16%	VS
 Argao, Jellloyd S.	- This member was pivotal in validating the entire system's functionality through rigorous testing and troubleshooting. They executed the docker-compose up command to build and start the multi-container application from scratch, monitoring the logs for any runtime errors or connection failures. They methodically tested all application features, verifying that frontend interactions correctly triggered API calls to the backend and that static assets were served without errors. When issues arose, such as failed connections between services, they diagnosed the problems within the Compose network and service configuration. They ensured that the application behaved consistently across different environments by testing on multiple systems. Their thorough validation confirmed the stability and reliability of the containerized solution before final deployment.	16%	VS

 Facun Jared Eizen B.	- This member took the lead in orchestrating the entire system by designing and writing the comprehensive docker-compose.yaml file. They defined all three services—backend, frontend, and nginx—and established the correct dependency chain to ensure services started in the proper order. A key contribution was architecting the named volumes and bind mounts to facilitate data persistence, specifically linking the built frontend files to the Nginx container's HTML directory. They configured the precise network relationships, ensuring all containers could communicate with each other using the service names as hostnames. They also mapped the host machine's ports to the Nginx container's port, making the application accessible from the outside world. This orchestration blueprint was the central piece that transformed the individual containers into a cohesive, functioning application.	18%	VS
 Piamonte, Malech Dangyan L.	- This member conducted essential research to inform the project's technical direction and resolve key dependencies. They investigated and compiled a list of the best, most secure, and lightweight base Docker images for Node.js and Nginx to serve as the foundation for the containers. They researched the correct syntax and structure for multi-stage Dockerfiles to optimize the frontend build process and researched the specific Nginx configuration directives needed for reverse proxying and serving static content. Furthermore, they studied Docker Compose syntax to understand how to properly define volumes, networks, and service dependencies for a complex setup. Their research provided the critical knowledge that guided the implementation decisions of the entire team, preventing common pitfalls and ensuring best practices were followed.	16%	VS
 Talavera, Adrian F.	- This member was responsible for preparing the frontend application for production within the Docker ecosystem. They engineered the process within the frontend's Dockerfile to build the static assets, which involved installing dependencies and executing the build command to generate optimized HTML, CSS, and JavaScript files. They strategically designed this container as a temporary build stage to keep the final images lean and secure. A significant contribution was their configuration of the frontend service within the docker-compose.yaml file, defining its build context and ensuring it depended on the healthy start of the backend. They verified that the frontend build process completed successfully and produced the correct artifacts for the web server. This work was crucial for creating a seamless pipeline from source code to production-ready static files.	20%	VS